

Transaction Management and Concurrency Control in DBMS

A Study Guide for Database Management Systems

1. What is a Transaction?

A transaction is a logical unit of work that consists of one or more database operations (reads and writes) that must be executed as a single, indivisible unit. The classic example is a bank transfer: debiting one account and crediting another must happen together or not at all.

A transaction begins with a BEGIN TRANSACTION statement and ends with either COMMIT (making all changes permanent) or ROLLBACK (undoing all changes). Between these boundaries, the database maintains the ability to undo all operations if anything goes wrong.

2. ACID Properties

Every transaction in a reliable database system must satisfy four fundamental properties, known by the acronym ACID:

Property	Definition	Enforced By
Atomicity	A transaction is all-or-nothing. Either all operations complete successfully, or none take effect.	Recovery Manager (Undo/Redo logs)
Consistency	A transaction transforms the database from one valid state to another. All integrity constraints are maintained.	Application logic + DB constraints
Isolation	Concurrent transactions appear to execute in isolation. Intermediate states are not visible to other transactions.	Concurrency Control (Locks, MVCC)
Durability	Once committed, changes survive system failures (crashes, power loss).	Write-Ahead Logging (WAL)

3. Transaction States

A transaction moves through several states during its lifecycle:

Active: The transaction is currently executing operations. This is the initial state after BEGIN TRANSACTION.

Partially Committed: The transaction has executed its final operation but has not yet been written to disk. At this point, the system checks if the commit can proceed safely.

Committed: All changes have been permanently written. The transaction cannot be rolled back.

Failed: The transaction encountered an error or was aborted. It must be rolled back.

Aborted: The transaction has been rolled back and the database is restored to its state prior to the transaction. The system may restart the transaction automatically or return an error to the user.

4. Schedules and Serializability

When multiple transactions execute concurrently, their operations are interleaved into a schedule. A schedule is serializable if its outcome is equivalent to some serial execution of the same transactions (one after another, no interleaving).

4.1 Conflict Serializability

Two operations conflict if they access the same data item and at least one is a write. A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting adjacent operations. This is tested using a precedence graph (serialization graph): if the graph is acyclic, the schedule is conflict-serializable.

4.2 View Serializability

A schedule is view-serializable if it is view-equivalent to a serial schedule. Two schedules are view-equivalent if: (1) they have the same initial reads, (2) the same write-read dependencies, and (3) the same final writes. View serializability is more permissive than conflict serializability but is NP-complete to test.

5. Concurrency Problems

Without proper concurrency control, several problems can arise when transactions execute simultaneously:

Problem	Description	Example
Dirty Read	Transaction reads data written by an uncommitted transaction.	T1 writes X=100, T2 reads X=100, T1 rollbacks. T2 has stale data.
Non-repeatable Read	Transaction reads same item twice and gets different values.	T1 reads X=50, T2 updates X=75 and commits, T1 reads X=75.
Phantom Read	Transaction re-executes a query and finds new rows inserted by another.	T1 counts rows WHERE dept="CS", T2 inserts a new CS row, T1 recounts.
Lost Update	Two transactions update same item; one update is overwritten.	T1 and T2 both read X=50. T1 writes X=60, T2 writes X=70. T1's update is lost.

6. Lock-Based Concurrency Control

The most common approach to concurrency control uses locks. A transaction must acquire a lock on a data item before accessing it.

6.1 Lock Types

Shared Lock (S-lock): Allows reading but not writing. Multiple transactions can hold shared locks on the same item simultaneously. Exclusive Lock (X-lock): Allows both reading and writing. Only one transaction can hold an exclusive lock on an item, and no shared locks can coexist with it.

	S-lock held	X-lock held
S-lock requested	Compatible	NOT Compatible
X-lock requested	NOT Compatible	NOT Compatible

6.2 Two-Phase Locking (2PL)

Two-Phase Locking is a protocol that guarantees conflict-serializable schedules. It has two phases: the Growing Phase, during which a transaction can acquire locks but cannot release any; and the Shrinking Phase, during which a transaction can release locks but cannot acquire new ones. The point where a transaction has acquired all its locks is called the lock point.

Variants of 2PL include Strict 2PL, where all exclusive locks are held until commit or abort (prevents cascading rollbacks); and Rigorous 2PL, where ALL locks (shared and exclusive) are held until commit or abort (simplifies recovery).

6.3 Deadlock

A deadlock occurs when two or more transactions are waiting for locks held by each other, creating a cycle of dependencies. For example: T1 holds lock on A and waits for B, while T2 holds lock on B and

waits for A.

Deadlock handling strategies include: Prevention (ordering resources or using timestamps with wait-die and wound-wait schemes), Detection (periodically checking for cycles in the wait-for graph and aborting a victim transaction), and Timeout (aborting transactions that wait too long for a lock).

7. SQL Isolation Levels

SQL defines four standard isolation levels that trade off between consistency and performance. Higher isolation means fewer anomalies but more overhead:

Isolation Level	Dirty Read	Non-repeatable	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Most production databases default to READ COMMITTED (PostgreSQL, Oracle) or REPEATABLE READ (MySQL/InnoDB). The SERIALIZABLE level provides the strongest guarantees but has the highest performance cost due to increased locking or validation overhead.

8. Recovery and Write-Ahead Logging

The Write-Ahead Logging (WAL) protocol ensures durability by requiring that log records are written to stable storage before the corresponding data modifications are written. This allows the recovery manager to redo committed transactions and undo uncommitted transactions after a crash.

Log records contain: the transaction ID, the data item modified, the old value (for undo), and the new value (for redo). During recovery, the system scans the log to identify which transactions committed (redo list) and which did not (undo list), then applies redo and undo operations as needed. Checkpointing periodically flushes dirty pages and records active transactions, reducing recovery time.